

# Human Computer Interaction

Janis Hutz  
<https://janishutz.com>

September 25, 2026



*“A funny quote by a professor”*

- Prof. Dr. Professor Name, YEAR

HS2026, ETHZ  
Summary of the course materials

Contents

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                      | <b>3</b> |
| <b>2</b> | <b>User Centered Design</b>              | <b>4</b> |
| 2.1      | Introduction . . . . .                   | 4        |
| 2.2      | Needfinding . . . . .                    | 5        |
| 2.2.1    | Why . . . . .                            | 5        |
| 2.2.2    | How . . . . .                            | 5        |
| 2.2.3    | Five Stages of Design Thinking . . . . . | 5        |
| <b>3</b> | <b>Prototyping</b>                       | <b>6</b> |
| <b>4</b> | <b>Evaluation</b>                        | <b>7</b> |

# 1 Introduction

This summary is likely to end up being quite a bit shorter than my other summaries due to the limited scope of this course, since its primary focus is the project and most of the theory covered is to be applied there, too.

## 2 User Centered Design

### 2.1 Introduction

To answer what User-Centred Design (UCD) is, we first need to establish what it *isn't*:

- UCD doesn't mean talking to the users and asking what they want (that's pretty much a user study)
- UCD doesn't mean testing the usability of the design in the end
- UCD is not just for user interfaces or software (it can also be for Programming Languages)
- UCD doesn't guarantee a perfect design (duh)

So, what is it then? It is characterized by:

1. **An early focus on users and tasks** (get a psychology degree first)
2. **Empirical measurement** (Come up with some sort of measurement of performance)
3. **Iterative Design** (Identify and fix problems)

That all is very obvious and the hardest part for Computer Scientists of course is the first property. The second property is typically done via user studies, where you keep track of a lot of data on how users interact with your application. Don't shy away from storing way more details than you think you could possibly use, it will make it much easier to evaluate things later on. (Can you tell that I am speaking from experience?)

That brings us to a formal definition from "The Design of Everyday Things" by Donald A. Norman (2013), which also includes famous examples such as the [Norman Door](#).

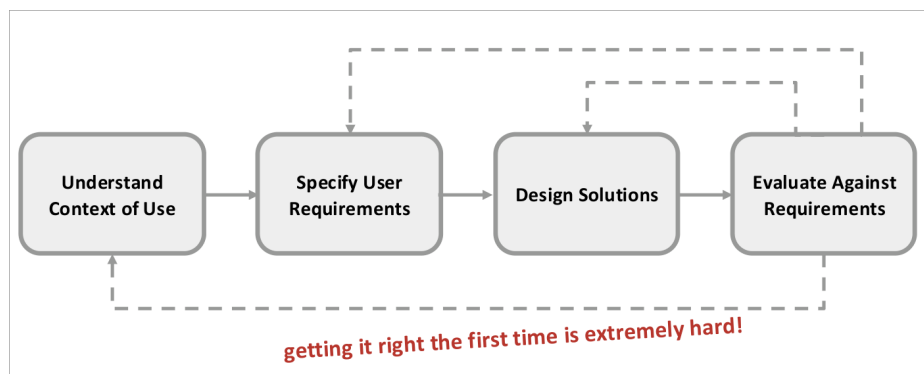
#### User-Centred Design

#### Definition 2.1.1

User-Centered Design (UCD) is an iterative design process in which designers focus on the users and their needs in each phase of the design process. UCD involves researching user behaviors, needs, and experiences and incorporating user feedback through multiple iterations of prototyping and testing to ensure usability, accessibility, and satisfaction. The goal is to develop solutions that align with real user needs rather than assumptions about those needs.

For property one and two, typically this is an iterating over *understanding the problem, building and evaluating*.

The full loop looks a like this:



Coming up with a good idea is hard, which is why this quote is important:

"The best way to have a good idea is to have *lots of ideas*"

— Linus Pauling

The only question that remains is, when are you done? See Section 4

## 2.2 Needfinding

Needfinding is the process of discovering what users *need*, not what they *want*. This is an important distinction, because most users can't actually tell you what they want unless presented with multiple options to choose from.

While it is a good idea to talk to users about what they are annoyed with in the system, asking them how they would change it is likely not going to give you much of an idea as to what you should change. Instead envision what could be possible solutions to the issues the users mentioned and implement mock-ups or prototypes for some of them and give those to actual users to try out, mentioning of course that this not a finished product.

This brings us to the distinction between **Needs** (human goals, i.e. what a user needs to achieve their task), **Requirements** (system goals, i.e. what the system needs to accomplish) and **Features** (solutions to the goals, but are not requirements by themselves).

Thus it is important to find out *why* the user does something in a certain way and *how* the user wants something and we know that we are doing the right thing.

### 2.2.1 Why

To find out why users do things a certain way is to ask basic questions of everyday experiences, but don't do that with closed-ended questions, such as "Do you like this design", and instead use something like "What do you like about the design and what would you like changed?". Remember to not ask *how* they want it changed, because that's not something users typically are good at.

### 2.2.2 How

How do we know if we are asking the right questions and if asking questions is even enough? Furthermore, two different people may interpret the same answers differently.

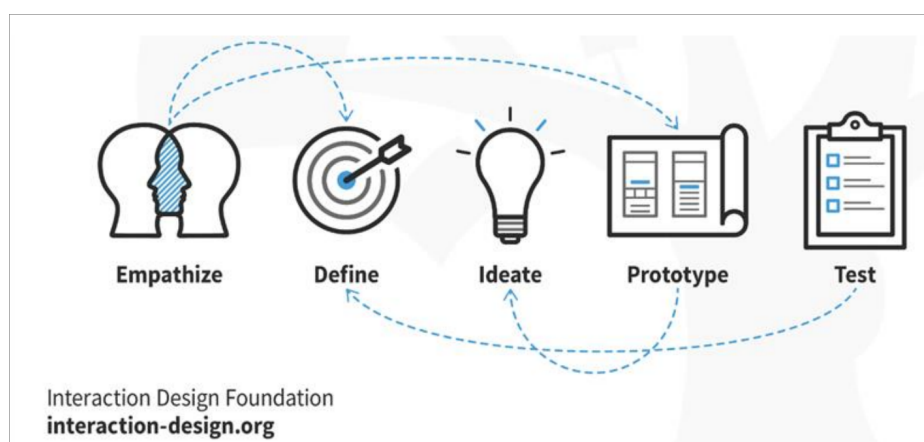
And finally, there are various biases:

- **Confirmation Bias:** The human brain tends to only consider facts that confirm ones view
- **Sampling Bias:** It is rarely possible to get a representative sample of all the users, which means that you will typically only get the opinions of the users that are easiest to reach.
- **Good Participant Bias:** Users tend to try to give positive or acceptable answers to please the researchers.
- **Hypothetical Bias:** Some users may say they would use a feature (in a certain way), but when actually having to use it, they won't.
- **Recency Bias:** Users focus on the most recent experience
- **Groupthink:** Design teams align too quickly on an interpretation without challenging it.

Common needfinding methods include:

- Interview
- Retrospective Study
- Contextual Inquiry
- Diary Study
- Artifact Analysis

### 2.2.3 Five Stages of Design Thinking



## 3 Prototyping

## 4 Evaluation